```
@events = Event.all
```

…with:

```
@events = Event.paginate(:per_page => 5, :page => params[:page],
:order => 'updated_at DESC')
```

Here, the parameters are as follows:

- *:per_page*: number of records on a page.
- *:page*: the number of the page to be fetched; the default is 1.
- *:order*: to decide on the ascending or descending order on a field.

Other parameters like *:conditions, :include*, etc, can also be used. For example, to list all events starting today, or in the future, you could add in a condition:

```
@events = Event.paginate(:per_page => 5, :page => params[:page],
 :order => 'DATE(start_date) asc', :conditions => ['start_date >=
?', Date.today])
```

In the index view, add the following code wherever you want the pagination links to appear:

```
# events/app/views/events/index.html.erb
<%= will_paginate %>
```

For further reading, check *http://rubydoc.info/gems/will_paginate/*.

Authlogic plug-in

Authlogic is an easy-to-use but powerful authentication plug-in for Rails. In laymen terms, it gives you user registration, sign-in (with a 'Remember me' option), sign-out and user account activation. Our rule is that only authenticated users can create an event.

Installation

Run the command to install the plugin:

```
script/plugin install git://github.com/binarylogic/authlogic.git
```

Adding user authentication to the application

The first thing that you need to do is associate the authentication functionality with the *User* model:

```
#user.rb
acts_as_authentic do |c|
 c.require_password_confirmation = false
end
```

In this tutorial, we are going to focus on user authentication only, and will create a new user using the *create* operation provided by the scaffold. We need to make one change to the view for new user -- the form should capture only email, login and password fields. Now, you must have noticed that our *User* model doesn't have a password field, so how can we use it in the new form? That's because the password is encrypted before it is stored—Authlogic takes the value of your password, encrypts it and stores it in the *crypted_password* field.

Once you associate a model with Authlogic, by default it adds a set of validations for the fields. Most of the fields are optional, but password confirmation is required by default; if you don't need it, then set it to false. You can read more about the other fields at the Authlogic ActsAsAuthentic documentation.

Create the *UserSession* model:

```
script/generate session user_session
```

Now create the *UserSessionsController*. Remember that the sign-in will not require the user to be signed in, while sign-out can happen only if the user is already signed in.

```
# events/app/controllers/user_sessions_controller.rb
before_filter :require_no_user, :only => [:new, :create]
before_filter :require_user, :only => :destroy
```

Add the following code to the *new* and *create* methods for the sign-in:

```
# events/app/controllers/user_sessions_controller.rb
 # new
  @user_session = UserSession.new

 # create
  @user_session = UserSession.new(params[:user_session])
   if @user_session.save
     flash[:notice] = "You have signed in successfully!"
     redirect_to root_url
   else
     render :action => :new
   end
```

For the sign-out, simply destroy the user session:

```
# app/controllers/user_sessions_controller.rb
 # destroy
   current_user_session.destroy
   flash[:notice] = "You have signed out successfully!"
   redirect_to root_url
```

In the view for the sign-in, the form should look like what follows:

```
# app/views/user_sessions/new.html.erb
<% form_for @user_session, :url => user_session_path do |f| %>
```